At-most once, at-least once, and exactly once

In modern architecture, systems are broken up into small and independent building blocks with well-defined interfaces between them. Message queues provide communication and coordination for those building blocks. Today, let's discuss different delivery semantics: at-most once, at-least once, and exactly once.

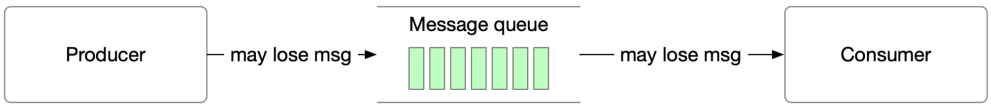


Figure 1 At-most once

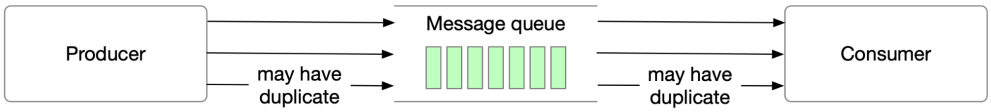


Figure 2 At-least once

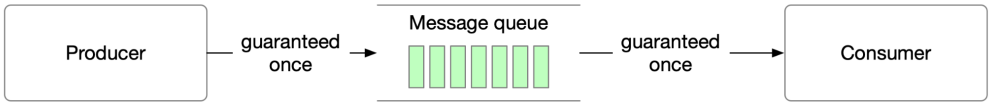


Figure 3 Exactly-once

At-most once

As the name suggests, at-most once means a message will be delivered not more than once. Messages may be lost but are not redelivered. This is how at-most once delivery works at the high level.

Use cases: It is suitable for use cases like monitoring metrics, where a small amount of data loss is acceptable.

At-least once

With this data delivery semantic, it's acceptable to deliver a message more than once, but no message should be lost.

Use cases: With at-least once, messages won't be lost but the same message might be delivered multiple times. While not ideal from a user perspective, at-least once delivery semantics are usually good enough for use cases where data duplication is not a big issue or deduplication

is possible on the consumer side. For example, with a unique key in each message, a message can be rejected when writing duplicate data to the database.

Exactly once

Exactly once is the most difficult delivery semantic to implement. It is friendly to users, but it has a high cost for the system's performance and complexity.

Use cases: Financial-related use cases (payment, trading, accounting, etc.). Exactly once is especially important when duplication is not acceptable and the downstream service or third party doesn't support idempotency.

Question: what is the difference between message queues vs event streaming platforms such as Kafka, Apache Pulsar, etc?

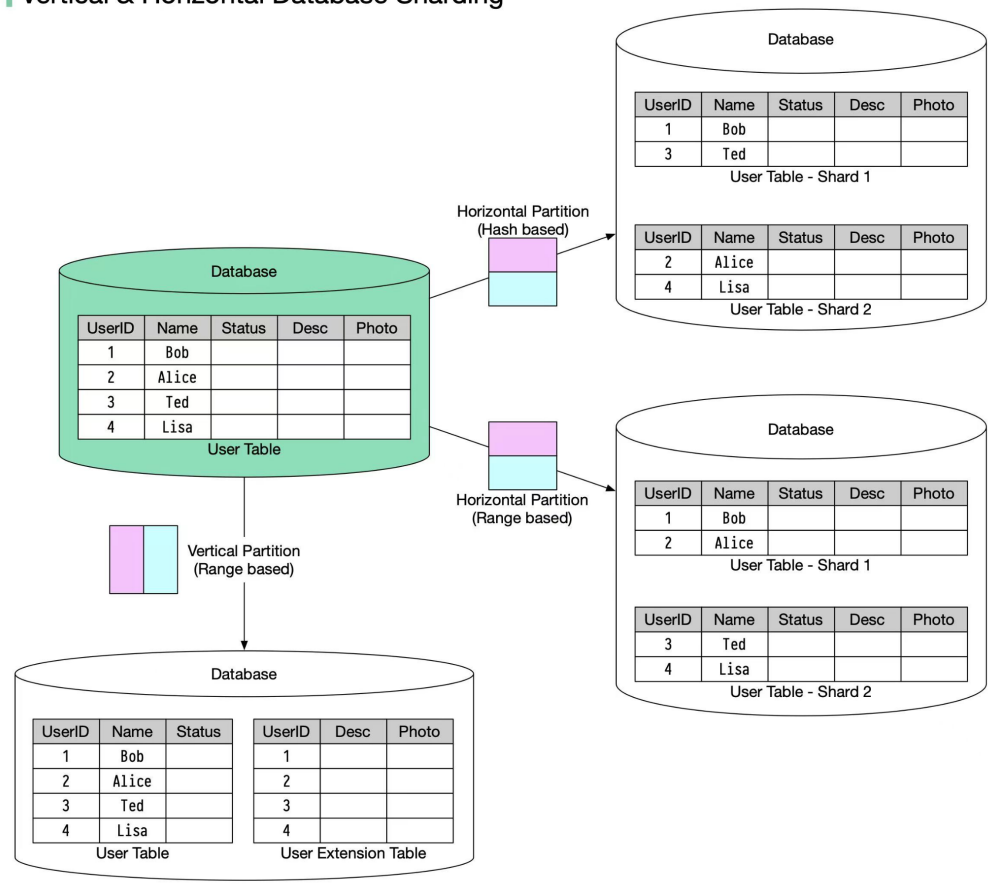
Vertical partitioning and Horizontal partitioning

In many large-scale applications, data is divided into partitions that can be accessed separately. There are two typical strategies for partitioning data.

- Vertical partitioning: it means some columns are moved to new tables. Each table contains the same number of rows but fewer columns (see diagram below).

- Horizontal partitioning (often called sharding): it divides a table into multiple smaller tables. Each table is a separate data store, and it contains the same number of columns, but fewer rows (see diagram below).

Vertical & Horizontal Database Sharding



Horizontal partitioning is widely used so let’s take a closer look.

Routing algorithm
Routing algorithm decides which partition (shard) stores the data.

🔹 Range-based sharding. This algorithm uses ordered columns, such as integers, longs, timestamps, to separate the rows. For example, the diagram below uses the User ID column for range partition: User IDs 1 and 2 are in shard 1, User IDs 3 and 4 are in shard 2.

🔹 Hash-based sharding. This algorithm applies a hash function to one column or several columns to decide which row goes to which table. For example, the diagram below uses **User ID mod** 2 as a hash function. User IDs 1 and 3 are in shard 1, User IDs 2 and 4 are in shard 2.

Benefits
🔹 Facilitate horizontal scaling. Sharding facilitates the possibility of adding more machines to spread out the load.

🔹 Shorten response time. By sharding one table into multiple tables, queries go over fewer rows, and results are returned much more quickly.

Drawbacks
🔹 The order by the operation is more complicated. Usually, we need to fetch data from different shards and sort the data in the application's code.

🔹 Uneven distribution. Some shards may contain more data than others (this is also called the hotspot).

This topic is very big and I’m sure I missed a lot of important details. What else do you think is important for data partitioning?

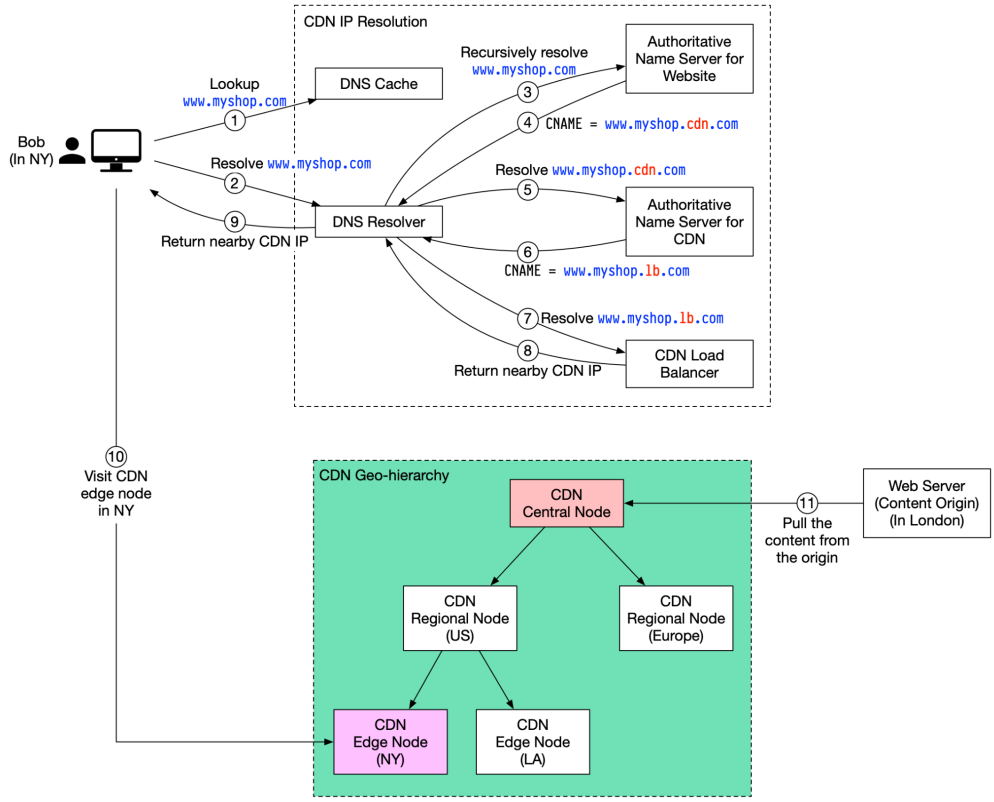
CDN

A content delivery network (CDN) refers to a geographically distributed servers (also called edge servers) which provide fast delivery of static and dynamic content. Let's take a look at how it works.

Suppose Bob who lives in New York wants to visit an eCommerce website that is deployed in London. If the request goes to servers located in London, the response will be quite slow. So we deploy CDN servers close to where Bob lives, and the content will be loaded from the nearby CDN server.

The diagram below illustrates the process:

How does CDN work



1. Bob types in www.myshop.com in the browser. The browser looks up the domain name in the local DNS cache.

2. If the domain name does not exist in the local DNS cache, the browser goes to the DNS resolver to resolve the name. The DNS resolver usually sits in the Internet Service Provider (ISP).

3. The DNS resolver recursively resolves the domain name (see my previous post for details). Finally, it asks the authoritative name server to resolve the domain name.

4. If we don’t use CDN, the authoritative name server returns the IP address for www.myshop.com. But with CDN, the authoritative name server has an alias pointing to www.myshop.cdn.com (the domain name of the CDN server).

5. The DNS resolver asks the authoritative name server to resolve www.myshop.cdn.com.

6. The authoritative name server returns the domain name for the load balancer of CDN www.myshop.lb.com.

7. The DNS resolver asks the CDN load balancer to resolve www.myshop.lb.com. The load balancer chooses an optimal CDN edge server based on the user’s IP address, user’s ISP, the content requested, and the server load.

8. The CDN load balancer returns the CDN edge server’s IP address for www.myshop.lb.com.

9. Now we finally get the actual IP address to visit. The DNS resolver returns the IP address to the browser.

10. The browser visits the CDN edge server to load the content. There are two types of contents cached on the CDN servers: static contents and dynamic contents. The former contains static pages, pictures, and videos; the latter one includes results of edge computing.

11. If the edge CDN server cache doesn't contain the content, it goes upward to the regional CDN server. If the content is still not found, it will go upward to the central CDN server, or even go to the origin - the

London web server. This is called the CDN distribution network, where the servers are deployed geographically.

Over to you: How do you prevent videos cached on CDN from being pirated?

Erasure coding

A really cool technique that's commonly used in object storage such as S3 to improve durability is called **Erasure Coding**. Let's take a look at how it works.

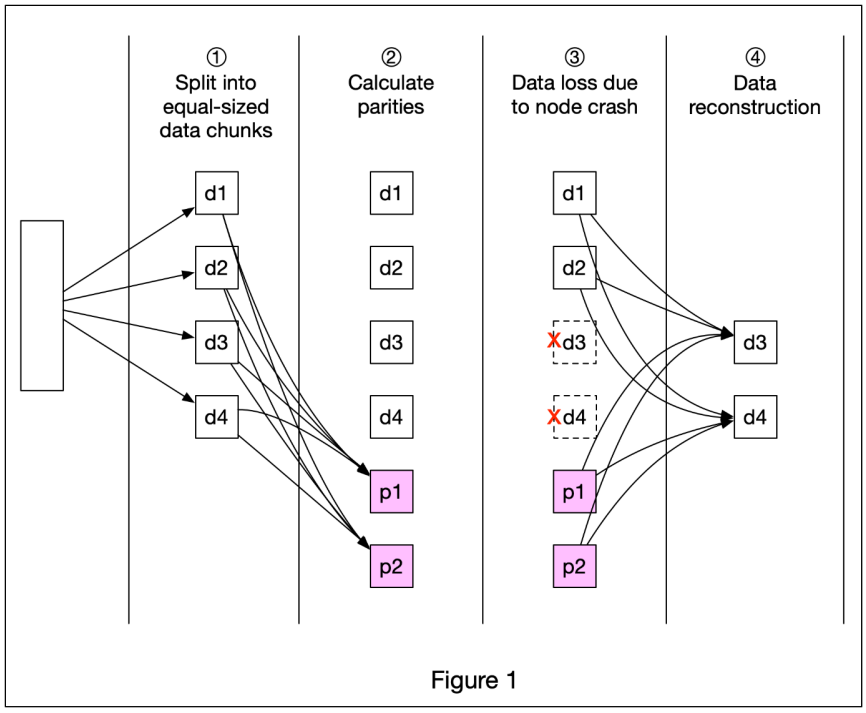


Figure 1

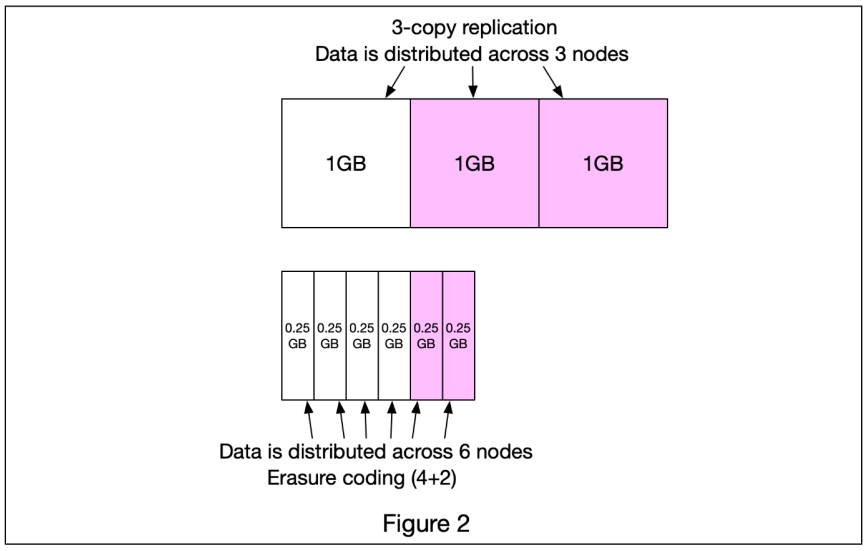


Figure 2

Erasure coding deals with data durability differently from replication. It chunks data into smaller pieces (placed on different servers) and creates parities for redundancy. In the event of failures, we can use chunk data and parities to reconstruct the data. Let's take a look at a concrete example (4 + 2 erasure coding) as shown in Figure 1.

1 Data is broken up into four even-sized data chunks d1, d2, d3, and d4.

2 The mathematical formula is used to calculate the parities p1 and p2. To give a much simplified example, p1 = d1 + 2*d2 - d3 + 4*d4 and p2 = -d1 + 5*d2 + d3 - 3*d4.

3 Data d3 and d4 are lost due to node crashes.

4 The mathematical formula is used to reconstruct lost data d3 and d4, using the known values of d1, d2, p1, and p2.

How much extra space does erasure coding need? For every two chunks of data, we need one parity block, so the storage overhead is 50% (Figure 2). While in 3-copy replication, the storage overhead is 200% (Figure 2).

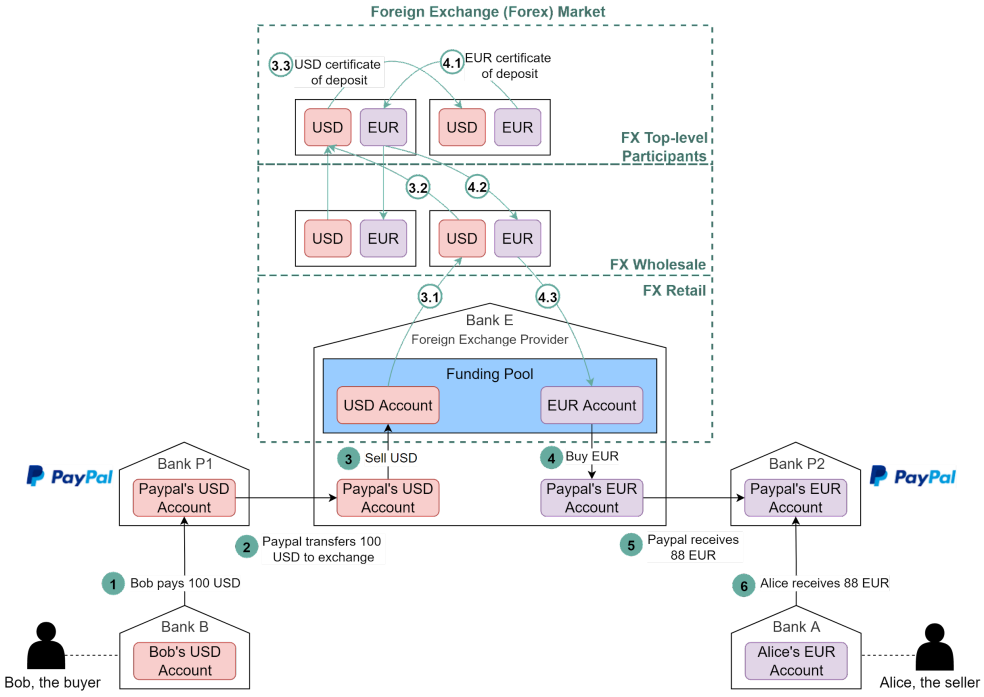
Does erasure coding increase data durability? Let's assume a node has a 0.81% annual failure rate. According to the calculation done by Backblaze, erasure coding can achieve 11 nines durability vs 3-copy replication can achieve 6 nines durability.

What other techniques do you think are important to improve the scalability and durability of an object store such as S3?

Foreign exchange in payment

Have you wondered what happens under the hood when you pay with USD online and the seller from Europe receives EUR (euro)? This process is called foreign exchange.

Foreign Exchange in Payments



Suppose Bob (the buyer) needs to pay 100 USD to Alice (the seller), and Alice can only receive EUR. The diagram below illustrates the process.

1. Bob sends 100 USD via a third-party payment provider. In our example, it is Paypal. The money is transferred from Bob's bank account (Bank B) to Paypal's account in Bank P1.

2. Paypal needs to convert USD to EUR. It leverages the foreign exchange provider (Bank E). Paypal sends 100 USD to its USD account in Bank E.

3. 100 USD is sold to Bank E's funding pool.

4. Bank E's funding pool provides 88 EUR in exchange for 100 USD. The money is put into Paypal's EUR account in Bank E.

5. Paypal's EUR account in Bank P2 receives 88 EUR.

6. 88 EUR is paid to Alice's EUR account in Bank A.

Now let's take a close look at the foreign exchange (forex) market. It has 3 layers:

- Retail market. Funding pools are parts of the retail market. To improve efficiency, Paypal usually buys a certain amount of foreign currencies in advance.
- Wholesale market. The wholesale business is composed of investment banks, commercial banks, and foreign exchange providers. It usually handles accumulated orders from the retail market.
- Top-level participants. They are multinational commercial banks that hold a large number of certificates of deposit from different countries. They exchange these certificates for foreign exchange trading.

When Bank E's funding pool needs more EUR, it goes upward to the wholesale market to sell USD and buy EUR. When the wholesale market accumulates enough orders, it goes upward to top-level participants. Steps 3.1-3.3 and 4.1-4.3 explain how it works.

If you have any questions, please leave a comment.

What foreign currency did you find difficult to exchange? And what company have you used for foreign currency exchange?

Interview Question: Design S3

What happens when you upload a file to Amazon S3? Let's design an S3 like object storage system.

Upload a File to S3

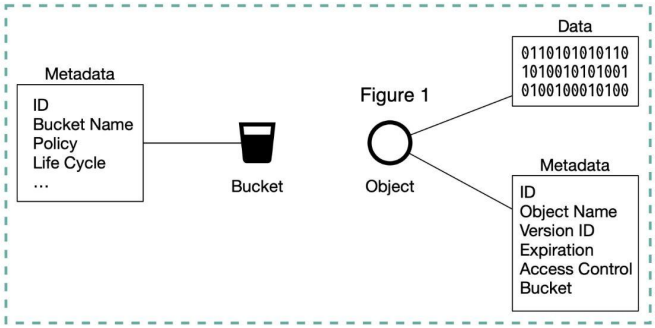


Figure 1

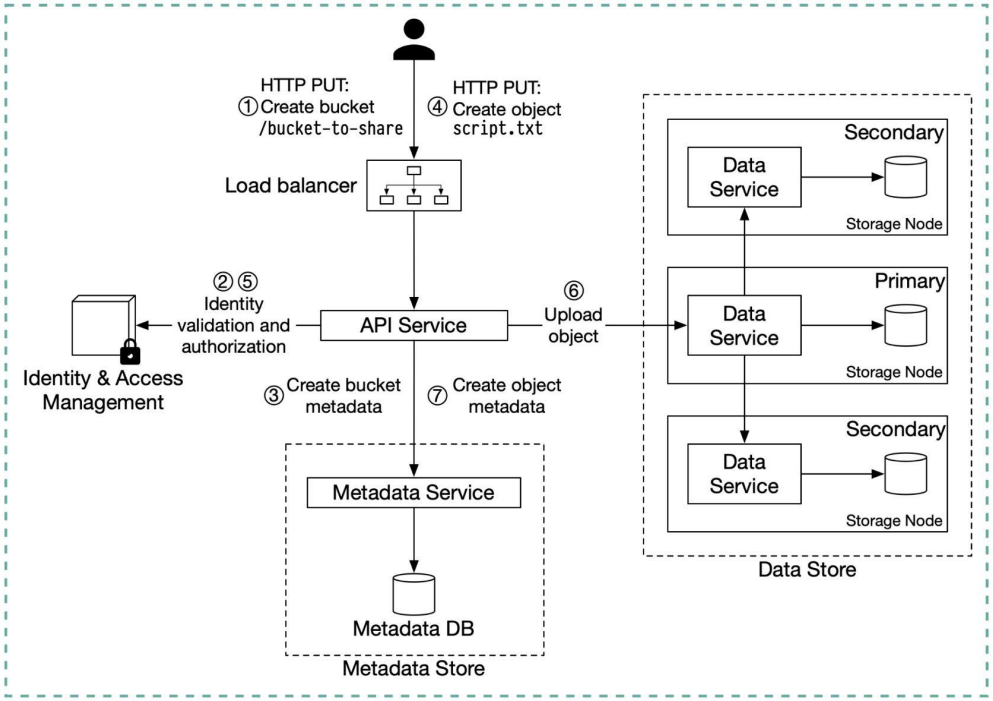


Figure 2

Before we dive into the design, let's define some terms.

Bucket. A logical container for objects. The bucket name is globally unique. To upload data to S3, we must first create a bucket.

Object. An object is an individual piece of data we store in a bucket. It contains object data (also called payload) and metadata. Object data can be any sequence of bytes we want to store. The metadata is a set of name-value pairs that describe the object.

An S3 object consists of (Figure 1):

- Metadata. It is mutable and contains attributes such as ID, bucket name, object name, etc.
- Object data. It is immutable and contains the actual data.

In S3, an object resides in a bucket. The path looks like this: /bucket-to-share/script.txt. The bucket only has metadata. The object has metadata and the actual data.

The diagram below (Figure 2) illustrates how file uploading works. In this example, we first create a bucket named “bucket-to-share” and then upload a file named “script.txt” to the bucket.

1. The client sends an HTTP PUT request to create a bucket named “bucket-to-share.” The request is forwarded to the API service.

2. The API service calls the Identity and Access Management (IAM) to ensure the user is authorized and has WRITE permission.

3. The API service calls the metadata store to create an entry with the bucket info in the metadata database. Once the entry is created, a success message is returned to the client.

4. After the bucket is created, the client sends an HTTP PUT request to create an object named “script.txt”.

5. The API service verifies the user’s identity and ensures the user has WRITE permission on the bucket.

6. Once validation succeeds, the API service sends the object data in the HTTP PUT payload to the data store. The data store persists the payload as an object and returns the UUID of the object.

7. The API service calls the metadata store to create a new entry in the metadata database. It contains important metadata such as the object_id (UUID), bucket_id (which bucket the object belongs to), object_name, etc.

Block storage, file storage and object storage

Yesterday, I posted the definitions of block storage, file storage, and object storage. Let's continue the discussion and compare those 3 options.

	Block storage	File storage	Object storage
Mutable Content	Y	Y	N (object versioning is supported, in-place update is not)
Cost	High	Medium to high	Low
Performance	Medium to high, very high	Medium to high	Low to medium
Consistency	Strong consistency	Strong consistency	Strong consistency
Data access	SAS/iSCSI/FC	Standard file access, CIFS/SMB, and NFS	RESTful API
Scalability	Medium scalability	High scalability	Vast scalability
Good for	Virtual machines (VM), high-performance applications like database	General-purpose file system access	Binary data, unstructured data

Table 1 Storage options

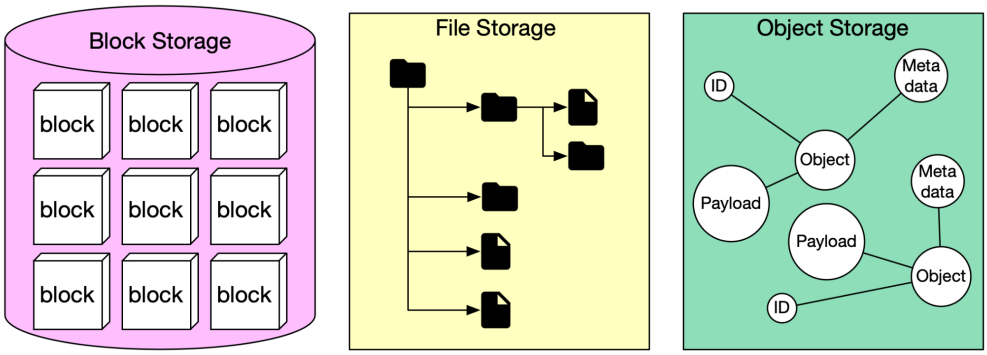
Block storage, file storage and object storage

In this post, let's review the storage systems in general.

Storage systems fall into three broad categories:

- Block storage
- File storage
- Object storage

The diagram below illustrates the comparison of different storage systems.



Block storage

Block storage came first, in the 1960s. Common storage devices like hard disk drives (HDD) and solid-state drives (SSD) that are physically attached to servers are all considered as block storage.

Block storage presents the raw blocks to the server as a volume. This is the most flexible and versatile form of storage. The server can format the raw blocks and use them as a file system, or it can hand control of those blocks to an application. Some applications like a database or a virtual machine engine manage these blocks directly in order to squeeze every drop of performance out of them.

Block storage is not limited to physically attached storage. Block storage could be connected to a server over a high-speed network or over industry-standard connectivity protocols like Fibre Channel (FC)

and iSCSI. Conceptually, the network-attached block storage still presents raw blocks. To the servers, it works the same as physically attached block storage. Whether to a network or physically attached, block storage is fully owned by a single server. It is not a shared resource.

File storage

File storage is built on top of block storage. It provides a higher-level abstraction to make it easier to handle files and directories. Data is stored as files under a hierarchical directory structure. File storage is the most common general-purpose storage solution. File storage could be made accessible by a large number of servers using common file-level network protocols like SMB/CIFS and NFS. The servers accessing file storage do not need to deal with the complexity of managing the blocks, formatting volume, etc. The simplicity of file storage makes it a great solution for sharing a large number of files and folders within an organization.

Object storage

Object storage is new. It makes a very deliberate tradeoff to sacrifice performance for high durability, vast scale, and low cost. It targets relatively “cold” data and is mainly used for archival and backup. Object storage stores all data as objects in a flat structure. There is no hierarchical directory structure. Data access is normally provided via a RESTful API. It is relatively slow compared to other storage types. Most public cloud service providers have an object storage offering, such as AWS S3, Google block storage, and Azure blob storage.

Domain Name System (DNS) lookup

DNS acts as an address book. It translates human-readable domain names (google.com) to machine-readable IP addresses (142.251.46.238).

To achieve better scalability, the DNS servers are organized in a hierarchical tree structure.

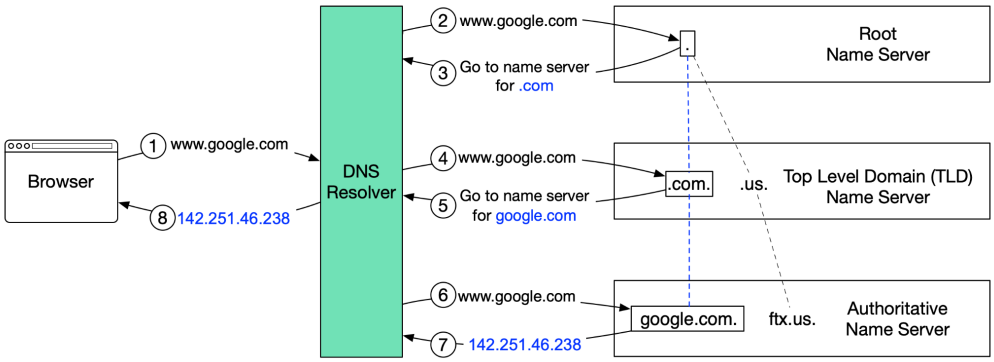
There are 3 basic levels of DNS servers:

1. Root name server (.). It stores the IP addresses of Top Level Domain (TLD) name servers. There are 13 logical root name servers globally.
2. TLD name server. It stores the IP addresses of authoritative name servers. There are several types of TLD names. For example, generic TLD (.com, .org), country code TLD (.us), test TLD (.test).

3. Authoritative name server. It provides actual answers to the DNS query. You can register authoritative name servers with domain name registrar such as GoDaddy, Namecheap, etc.

The diagram below illustrates how DNS lookup works under the hood:

How does DNS resolve IP



1. google.com is typed into the browser, and the browser sends the domain name to the DNS resolver.

2. The resolver queries a DNS root name server.

3. The root server responds to the resolver with the address of a TLD DNS server. In this case, it is .com.

4. The resolver then makes a request to the .com TLD.

5. The TLD server responds with the IP address of the domain's name server, google.com (authoritative name server).

6. The DNS resolver sends a query to the domain's nameserver.

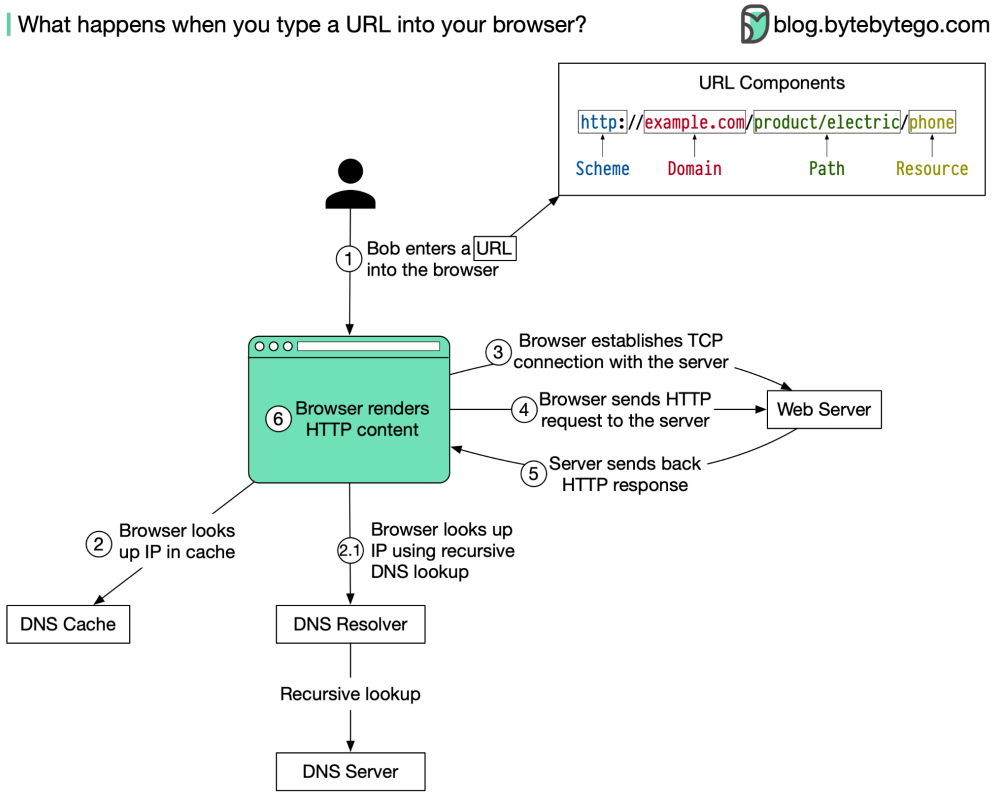
7. The IP address for google.com is then returned to the resolver from the nameserver.

8. The DNS resolver responds to the web browser with the IP address (142.251.46.238) of the domain requested initially.

DNS lookups on average take between 20-120 milliseconds to complete (according to YSlow).

What happens when you type a URL into your browser?

The diagram below illustrates the steps.



1. Bob enters a URL into the browser and hits Enter. In this example, the URL is composed of 4 parts:

- scheme - ***https***://. This tells the browser to send a connection to the server using HTTPS.
- domain - ***example.com***. This is the domain name of the site.
- path - ***product/electric***. It is the path on the server to the requested resource: phone.
- resource - ***phone***. It is the name of the resource Bob wants to visit.

2. The browser looks up the IP address for the domain with a domain name system (DNS) lookup. To make the lookup process fast, data is cached at different layers: browser cache, OS cache, local network cache and ISP cache.

2.1 If the IP address cannot be found at any of the caches, the browser goes to DNS servers to do a recursive DNS lookup until the IP address is found (this will be covered in another post).

3. Now that we have the IP address of the server, the browser establishes a TCP connection with the server.

4. The browser sends a HTTP request to the server. The request looks like this:

```
GET /phone HTTP/1.1
Host: example.com
```

5. The server processes the request and sends back the response. For a successful response (the status code is 200). The HTML response might look like this:

```
HTTP/1.1 200 OK
Date: Sun, 30 Jan 2022 00:01:01 GMT
Server: Apache
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="en">
Hello world
</html>
```

6. The browser renders the HTML content.

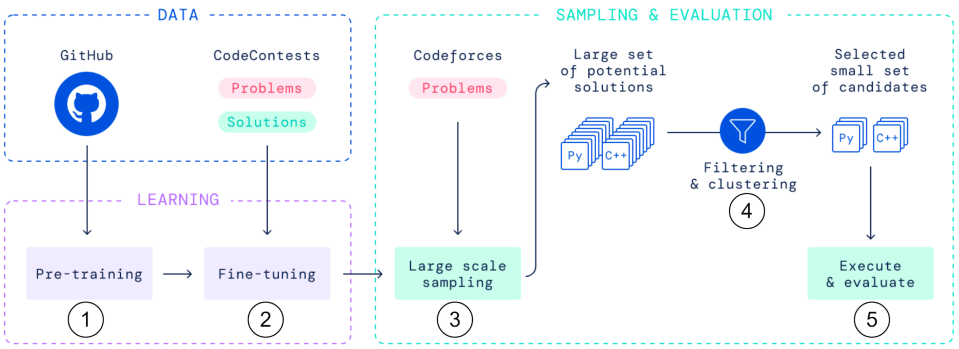
AI Coding engine

DeepMind says its new AI coding engine (AlphaCode) is as good as an average programmer.

The AI bot participated in the 10 Codeforces coding competitions and was ranked 54.3%. It means its score exceeded half of the human contestants. If we look at its score for the last 6 months, AlphaCode ranks at 28%.

The diagram below explains how the AI bot works:

> How does AlphaCode Work?



Source: DeepMind Blog

1. Pre-train the transformer models on GitHub code.

2. Fine-tune the models on the relatively small competitive programming dataset.

3. At evaluation time, create a massive amount of solutions for each problem.

4. Filter, cluster and rerank the solutions to a small set of candidate programs (at most 10), and then submit for further assessments.

5. Run the candidate programs against the test cases, evaluate the performance, and choose the best one.

Do you think AI bot will be better at Leetcode or competitive programming than software engineers five years from now?

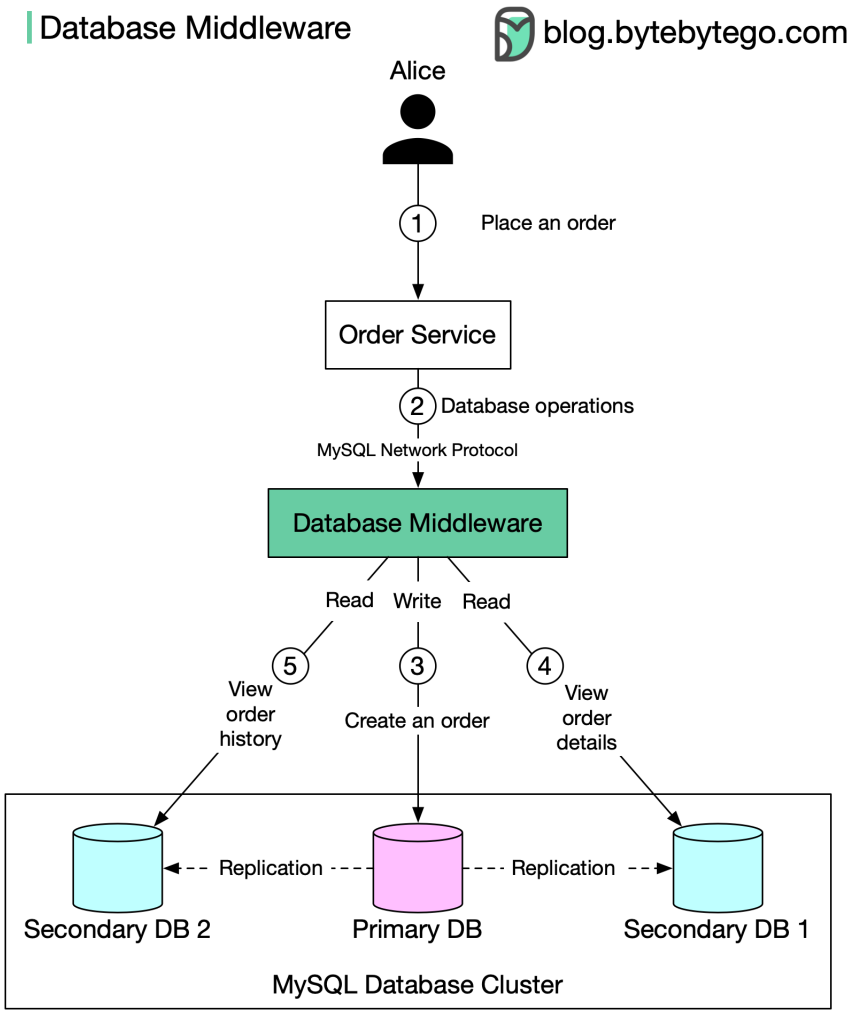
Read replica pattern

There are two common ways to implement the read replica pattern:

1. Embed the routing logic in the application code (explained in the last post).
2. Use database middleware.

We focus on option 2 here. The middleware provides transparent routing between the application and database servers. We can customize the routing logic based on difficult rules such as user, schema, statement, etc.

The diagram below illustrates the setup:



1. When Alice places an order on amazon, the request is sent to Order Service.

2. Order Service does not directly interact with the database. Instead, it sends database queries to the database middleware.

3. The database middleware routes writes to the primary database. Data is replicated to two replicas.

4. Alice views the order details (read). The request is sent through the middleware.

5. Alice views the recent order history (read). The request is sent through the middleware.

The database middleware acts as a proxy between the application and databases. It uses standard MySQL network protocol for communication.

Pros:
- Simplified application code. The application doesn't need to be aware of the database topology and manage access to the database directly.

- Better compatibility. The middleware uses the MySQL network protocol. Any MySQL compatible client can connect to the middleware easily. This makes database migration easier.

Cons:
- Increased system complexity. A database middleware is a complex system. Since all database queries go through the middleware, it usually requires a high availability setup to avoid a single point of failure.

- Additional middleware layer means additional network latency. Therefore, this layer requires excellent performance.

Read replica pattern

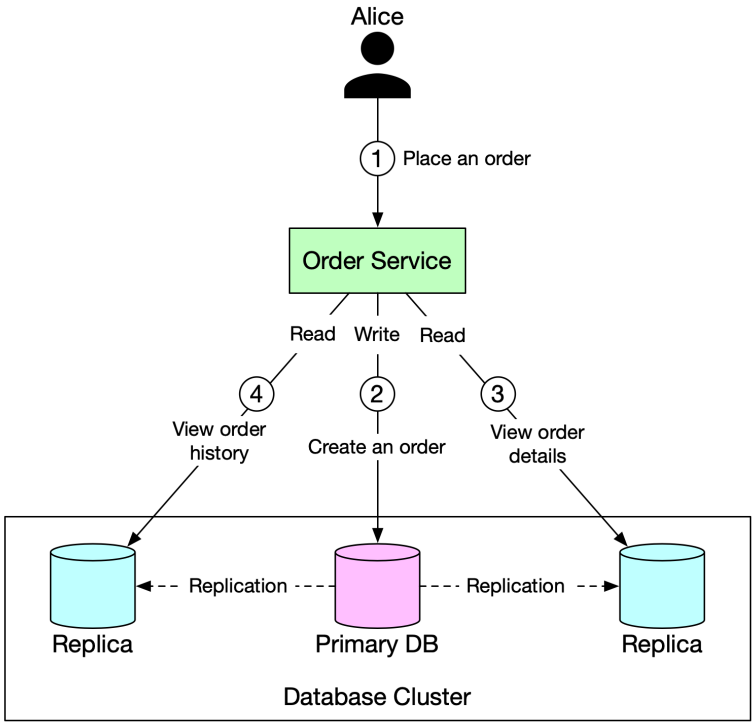
In this post, we talk about a simple yet commonly used database design pattern (setup): **Read replica pattern**.

In this setup, all data-modifying commands like insert, delete, or update are sent to the primary DB, and reads are sent to read replicas.

The diagram below illustrates the setup:

1. When Alice places an order on amazon.com, the request is sent to Order Service.
2. Order Service creates a record about the order in the primary DB (write). Data is replicated to two replicas.
3. Alice views the order details. Data is served from a replica (read).
4. Alice views the recent order history. Data is served from a replica (read).

Read Replica Pattern



There is one major problem in this setup: **replication lag**.

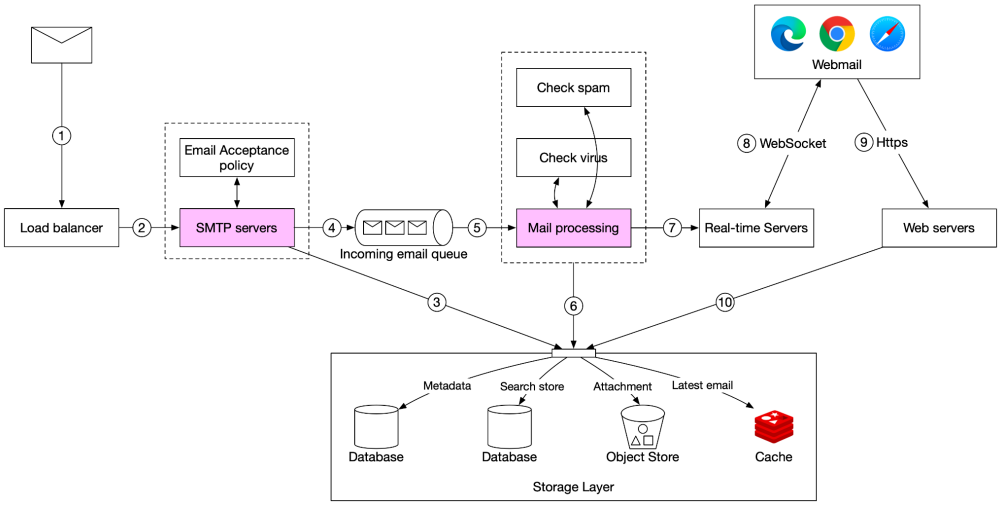
Under certain circumstances (network delay, server overload, etc.), data in replicas might be seconds or even minutes behind. In this case, if Alice immediately checks the order status (query is served by the replica) after the order is placed, she might not see the order at all. This leaves Alice confused. In this case, we need “read-after-write” consistency.

Possible solutions to mitigate this problem:

1. Latency sensitive reads are sent to the primary database.
2. Reads that immediately follow writes are routed to the primary database.
3. A relational DB generally provides a way to check if a replica is caught up with the primary. If data is up to date, query the replica. Otherwise, fail the read request or read from the primary.

Email receiving flow

The following diagram demonstrates the email receiving flow.



1. Incoming emails arrive at the SMTP load balancer.

2. The load balancer distributes traffic among SMTP servers. Email acceptance policy can be configured and applied at the SMTP-connection level. For example, invalid emails are bounced to avoid unnecessary email processing.

3. If the attachment of an email is too large to put into the queue, we can put it into the attachment store (s3).

4. Emails are put in the incoming email queue. The queue decouples mail processing workers from SMTP servers so they can be scaled independently. Moreover, the queue serves as a buffer in case the email volume surges.

5. Mail processing workers are responsible for a lot of tasks, including filtering out spam mails, stopping viruses, etc. The following steps assume an email passed the validation.

6. The email is stored in the mail storage, cache, and object data store.

7. If the receiver is currently online, the email is pushed to real-time servers.

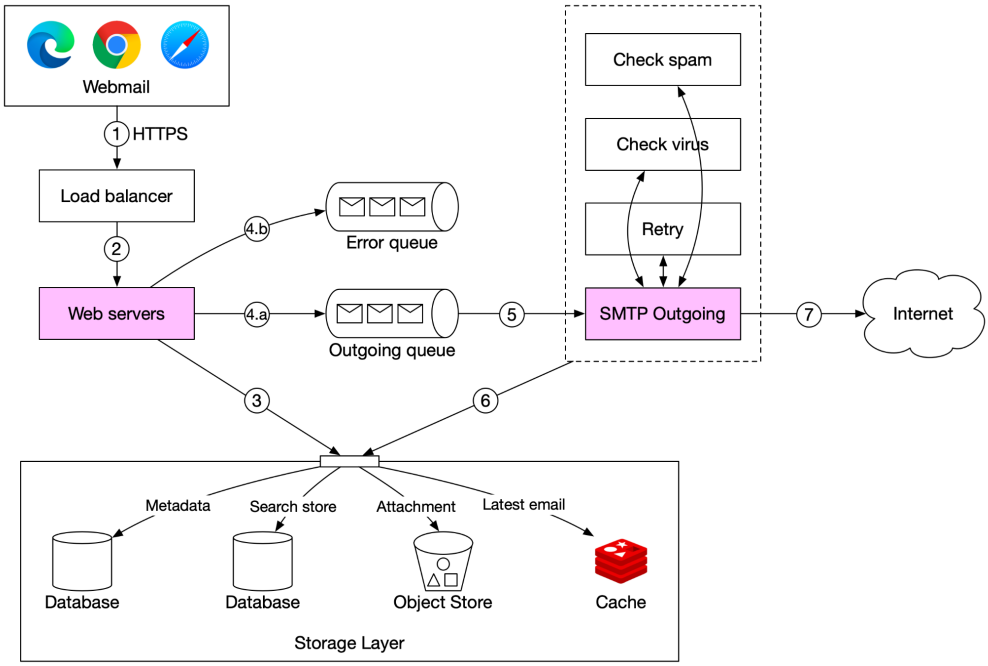
8. Real-time servers are WebSocket servers that allow clients to receive new emails in real-time.

9. For offline users, emails are stored in the storage layer. When a user comes back online, the webmail client connects to web servers via RESTful API.

10. Web servers pull new emails from the storage layer and return them to the client.

Email sending flow

In this post, we will take a closer look at the email sending flow.



1. A user writes an email on webmail and presses the “send” button. The request is sent to the load balancer.

2. The load balancer makes sure it doesn’t exceed the rate limit and routes traffic to web servers.

3. Web servers are responsible for:

- Basic email validation. Each incoming email is checked against pre-defined rules such as email size limit.

- Checking if the domain of the recipient’s email address is the same as the sender. If it is the same, email data is inserted to storage, cache, and object store directly. The recipient can fetch the email directly via the RESTful API. There is no need to go to step 4.

4. Message queues.

4.a. If basic email validation succeeds, the email data is passed to the outgoing queue.

4.b. If basic email validation fails, the email is put in the error queue.

5. SMTP outgoing workers pull events from the outgoing queue and make sure emails are spam and virus free.

6. The outgoing email is stored in the “Sent Folder” of the storage layer.

7. SMTP outgoing workers send the email to the recipient mail server.

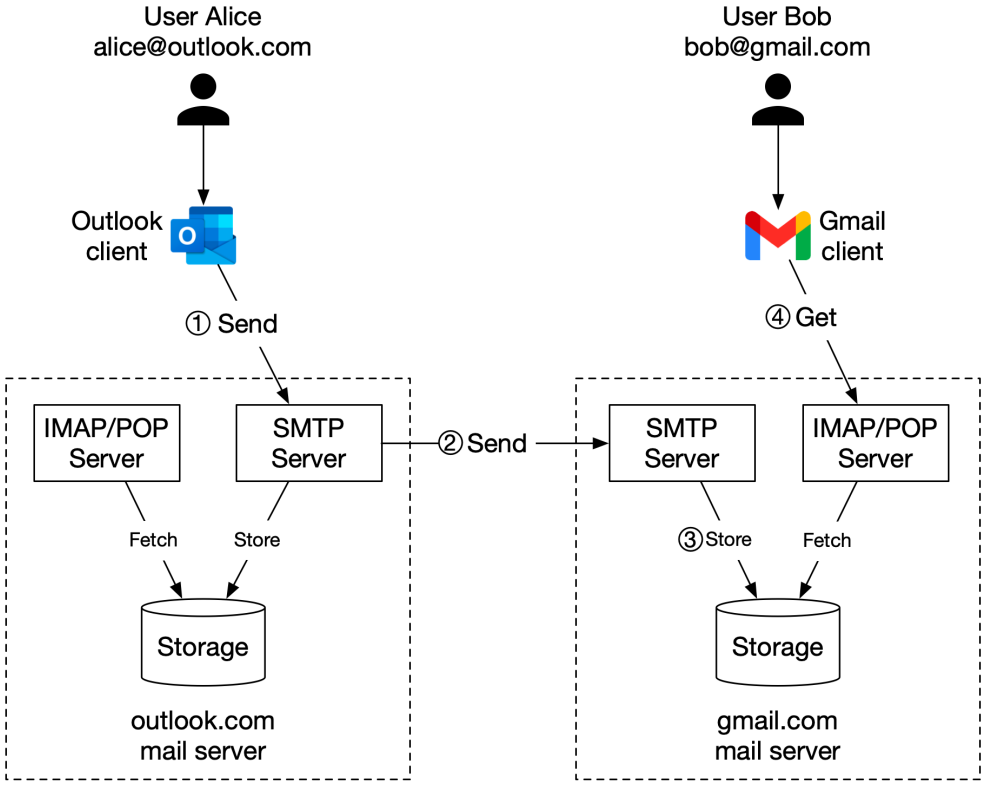
Each message in the outgoing queue contains all the metadata required to create an email. A distributed message queue is a critical component that allows asynchronous mail processing. By decoupling SMTP outgoing workers from the web servers, we can scale SMTP outgoing workers independently.

We monitor the size of the outgoing queue very closely. If there are many emails stuck in the queue, we need to analyze the cause of the issue. Here are some possibilities:
- The recipient’s mail server is unavailable. In this case, we need to retry sending the email at a later time. Exponential backoff might be a good retry strategy.

- Not enough consumers to send emails. In this case, we may need more consumers to reduce the processing time.

Interview Question: Design Gmail

One picture is worth more than a thousand words. In this post, we will take a look at what happens when Alice sends an email to Bob.



1. Alice logs in to her Outlook client, composes an email, and presses “send”. The email is sent to the Outlook mail server. The communication protocol between the Outlook client and mail server is SMTP.

2. Outlook mail server queries the DNS (not shown in the diagram) to find the address of the recipient’s SMTP server. In this case, it is Gmail’s SMTP server. Next, it transfers the email to the Gmail mail server. The communication protocol between the mail servers is SMTP.

3. The Gmail server stores the email and makes it available to Bob, the recipient.

4. Gmail client fetches new emails through the IMAP/POP server when Bob logs in to Gmail.

Please keep in mind this is a highly simplified design. Hope it sparks your interest and curiosity:) I'll explain each component in more depth in the future.

Map rendering

Google Maps Continued. Let's take a look at **Map Rendering** in this post.

Pre-Computed Tiles

One foundational concept in map rendering is tiling. Instead of rendering the entire map as one large custom image, the world is broken up into smaller tiles. The client only downloads the relevant tiles for the area the user is in and stitches them together like a mosaic for display. The tiles are pre-computed at different zoom levels. Google Maps uses 21 zoom levels.

For example, at zoom level 0, The entire map is represented by a single tile of size 256 * 256 pixels. Then at zoom level 1, the number of map tiles doubles in both north-south and east-west directions, while each tile stays at 256 * 256 pixels. So we have 4 tiles at zoom level 1, and the whole image of zoom level 1 is 512 * 512 pixels. With each increment, the entire set of tiles has 4x as many pixels as the previous level. The increased pixel count provides an increasing level of detail to the user.

This allows the client to render the map at the best granularities depending on the client's zoom level without consuming excessive bandwidth to download tiles with too much detail. This is especially important when we are loading the images from mobile clients.

Road Segments

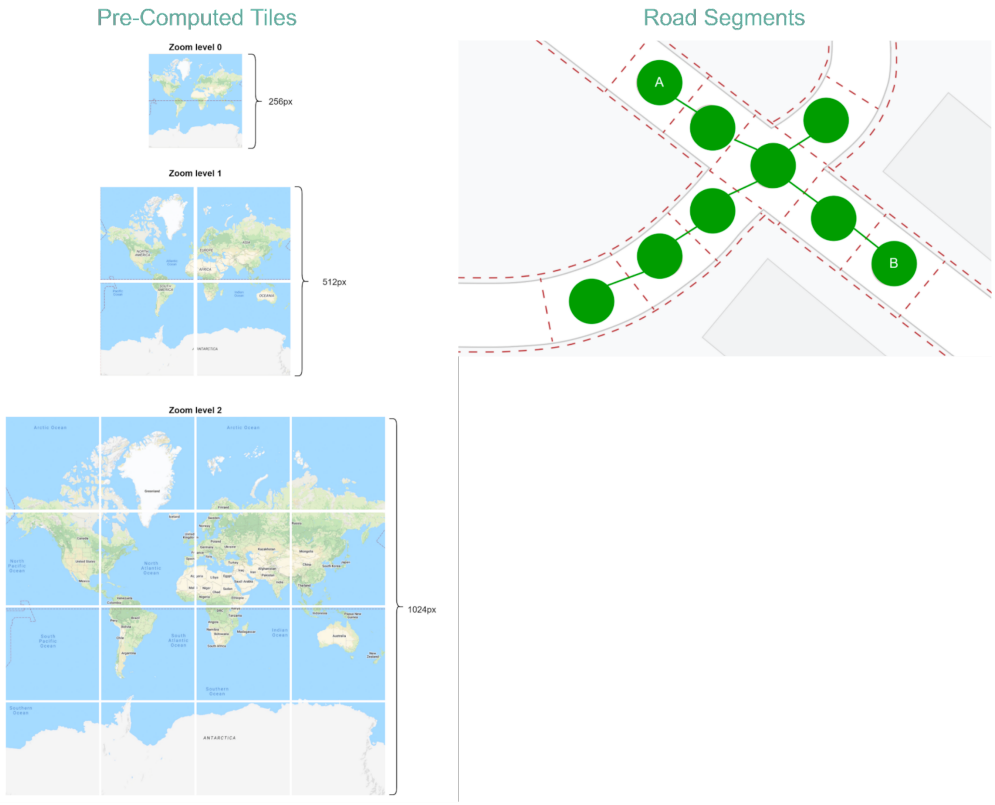
Now that we have transformed massive maps into tiles, we also need to define a data structure for the roads. We divide the world of roads into small blocks. We call these blocks road segments. Each road segment contains multiple roads, junctions, and other metadata.

We group nearby segments into super segments. This process can be applied repeatedly to meet the level of coverage required.

We then transform the road segments into a data structure that the navigation algorithms can use. The typical approach is to convert the map into a ***graph***, where the nodes are road segments, and two nodes are connected if the corresponding road segments are reachable

neighbors. In this way, finding a path between two locations becomes a shortest-path problem, where we can leverage Dijkstra or A* algorithms.

Google Maps - Map Rendering

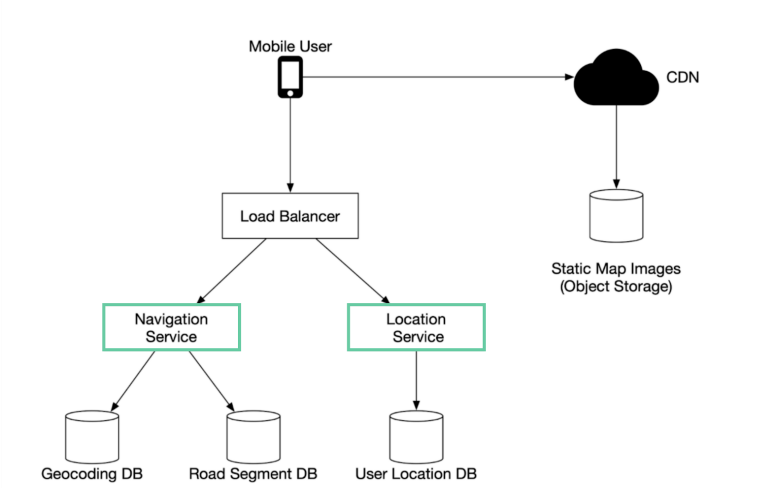


Interview Question: Design Google Maps

Google started project Google Maps in 2005. As of March 2021, Google Maps had one billion daily active users, 99% coverage of the world in 200 countries.

Although Google Maps is a very complex system, we can break it down into 3 high-level components. In this post, let's take a look at how to design a simplified Google Maps.

Google Maps



2D Map Projection

Location Service

The location service is responsible for recording a user's location update. The Google Map clients send location updates every few seconds. The user location data is used in many cases:

- detect new and recently closed roads
- improve the accuracy of the map over time
- used as an input for live traffic data.

Map Rendering

The world's map is projected into a huge 2D map image. It is broken down into small image blocks called "tiles" (see below). The tiles are static. They don't change very often. An efficient way to serve static tile files is with a CDN backed by cloud storage like S3. The users can load the necessary tiles to compose a map from nearby CDN.

What if a user is zooming and panning the map viewpoint on the client to explore their surroundings?

An efficient way is to pre-calculate the map blocks with different zoom levels and load the images when needed.

Navigation Service

This component is responsible for finding a reasonably fast route from point A to point B. It calls two services to help with the path calculation:

1. Geocoding Service: resolve the given address to a latitude/longitude pair

2. Route Planner Service: this service does three things in sequence:

    - Calculate the top-K shortest paths between A and B
    - Calculate the estimation of time for each path based on current traffic and historical data
    - Rank the paths by time predictions and user filtering. For example, the user doesn't want to avoid tolls.

Pull vs push models

There are two ways metrics data can be collected, pull or push. It is a routine debate as to which one is better and there is no clear answer. In this post, we will take a look at the pull model.

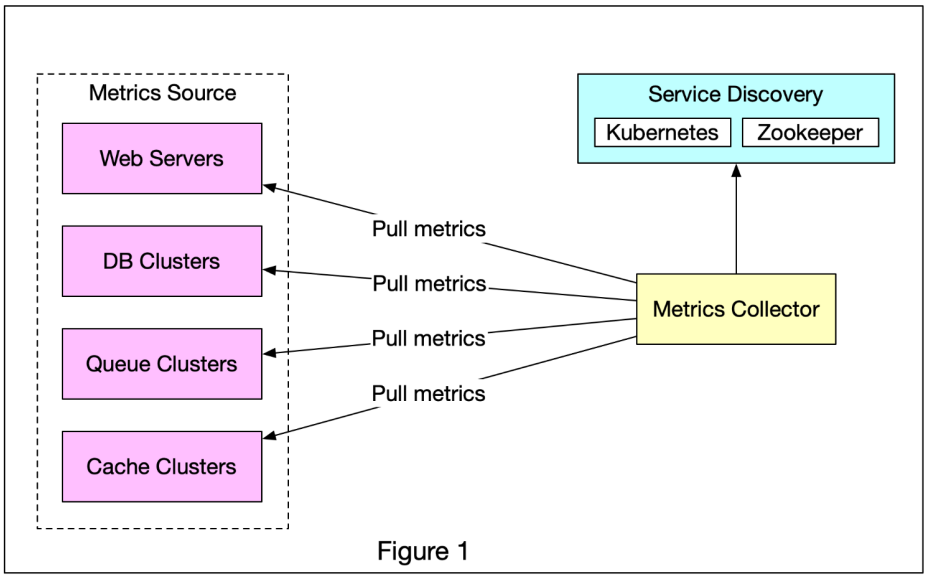


Figure 1

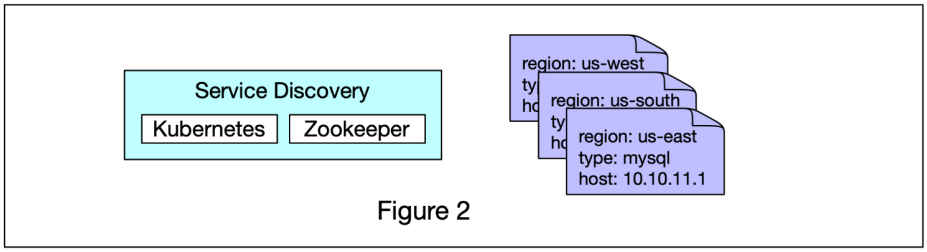


Figure 2

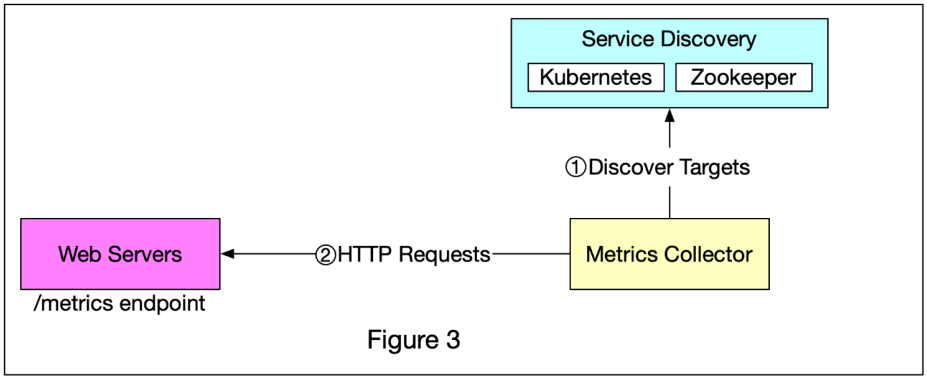


Figure 3

Figure 1 shows data collection with a pull model over HTTP. We have dedicated metric collectors which pull metrics values from the running applications periodically.

In this approach, the metrics collector needs to know the complete list of service endpoints to pull data from. One naive approach is to use a file to hold DNS/IP information for every service endpoint on the “metric collector” servers. While the idea is simple, this approach is hard to maintain in a large-scale environment where servers are added or removed frequently, and we want to ensure that metric collectors don’t miss out on collecting metrics from any new servers.

The good news is that we have a reliable, scalable, and maintainable solution available through Service Discovery, provided by Kubernetes, Zookeeper, etc., wherein services register their availability and the metrics collector can be notified by the Service Discovery component whenever the list of service endpoints changes. Service discovery contains configuration rules about when and where to collect metrics as shown in Figure 2.

Figure 3 explains the pull model in detail.

1 The metrics collector fetches configuration metadata of service endpoints from Service Discovery. Metadata include pulling interval, IP addresses, timeout and retries parameters, etc.

2 The metrics collector pulls metrics data via a pre-defined HTTP endpoint (for example, /metrics). To expose the endpoint, a client library usually needs to be added to the service. In Figure 3, the service is Web Servers.

3 Optionally, the metrics collector registers a change event notification with Service Discovery to receive an update whenever the service endpoints change. Alternatively, the metrics collector can poll for endpoint changes periodically.

Money movement

One picture is worth more than a thousand words. This is what happens when you buy a product using Paypal/bank card under the hood.

To understand this, we need to digest two concepts: **clearing** & **settlement**. Clearing is a process that calculates who should pay whom with how much money; while settlement is a process where real money moves between reserves in the settlement bank.

Money Movement